

Руководство по алгоритмам DataShield

Алгоритмический справочник по кодеку DataShield: формат пакета, логика кодирования и декодирования, математика избыточности, накопление секторов, адресная перепривязка и сборка файла с обходом коллизий версий, рандомизированный тестовый стенд. Документ описывает только алгоритмы и структуры данных; публичный API и структура решения описываются в `DeveloperGuide.ru.md`. Каждая из трёх задач раскрыта отдельным семейством документов в трёх уровнях детализации (строгий академический, инженерный пересказ с совпадающей нумерацией разделов, вводное учебное пособие «с нуля»): кодирование — `EncoderGuide.Academic/Engineer/Tutorial.ru.md`, декодирование — `DecoderGuide.Academic/Engineer/Tutorial.ru.md`, сборка и коллизии — `AssemblyGuide.Academic/Engineer/Tutorial.ru.md` (инварианты, математика перебора и прокрутки, проработанные примеры, границы гарантий).

1. Назначение и принципы

DataShield — кодек прямой коррекции ошибок (FEC) для передачи небольших файлов через ненадёжные однонаправленные каналы: текстовые пасты, мессенджеры, тела писем, сырые байтовые потоки. Файл превращается в поток маленьких самодостаточных пакетов; декодер собирает файл из произвольно повреждённого, переставленного, дублированного и многофайлового потока.

Цель	Механизм
Транспорт через текст	Пакет = 75 байт = ровно 100 символов Base64 без паддинга
Устойчивость к потерям	Стирающий код Рида–Соломона над GF(2 ¹⁶) на уровне томов
Произвольный порядок прихода	Накопительный приём без порядка; сектор несёт только свой номер
Устойчивость к шуму	Сканирование скользящим окном с усечённым SHA-256 на каждый пакет
Многофайловые потоки	Привязка сектора к заголовку через хеш с сидом H5
Подделки и коллизии	Версии payload со счётчиками подтверждений + поиск правильной комбинации при сборке

Ключевая идея проектирования: **ни один байт входного потока не считается мусором, пока он не проверен криптографически стойкой контрольной суммой**. Всё остальное — следствия этой идеи.

2. Формат пакета

Все пакеты имеют одинаковый размер — 75 байт (`PacketFormat.PacketSize`). В текстовом режиме пакет кодируется Base64: $75 \cdot 4/3 =$ ровно **100 символов без паддинга**, одна строка — один пакет.

2.1. Пакет заголовка (H)

Поле	Смещение	Размер	Содержимое
H1	0	14	Имя файла (упакованное, см. ниже), ASCII, дополненное пробелами
H2	14	3	Размер файла, младшие 3 байта UInt32, LE
H3	17	32	SHA-256 содержимого файла
H4	49	2	Число ECC-томов M, UInt16 LE
H5	51	24	Trunc24(SHA-256(H1–H4)) — 192 бита

Поле H1 хранит не полное имя, а упакованное (`FileNameCodec.Pack`): имя разбивается по **первой** точке, расширение (включая составные, вида «.tar.gz») сохраняется целиком, а база усекается с маркером «~» на конце (`documents.tar.gz` → `docume~.tar.gz`). Если на базу остаётся меньше 2 байт — имя не представимо, кодирование отклоняется. Полное имя восстанавливается по SHA-256 (H3) владельцем файла.

2.2. Сектор данных (D)

Поле	Смещение	Размер	Содержимое
D1	0	2	Номер сектора (тома), UInt16 LE
D2	2	64	Payload — данные файла или ECC
D3	66	9	Trunc9(SHA-256(H5 D1 D2)) — 72 бита

2.3. Два уровня хешей — сердце формата

- **Хеш заголовка (H5) автономен.** Trunc24(SHA-256(H1–H4)) проверяется без какого-либо контекста. Это позволяет опознать заголовок в шумном потоке «с нуля» (вероятность ложного срабатывания $\sim 2^{-192}$).
- **Хеш сектора (D3) привязан к заголовку.** Он вычисляется как Trunc9(SHA-256(H5 | D1 | D2)): сидом служит H5 заголовка-владельца. Без знания H5 проверка невозможна — случайные 75 байт почти наверняка не пройдут её ($\sim 2^{-72}$).

Следствия:

1. Сектор **криптографически привязан к своему файлу**. В многофайловом потоке сектор не перепутается с чужим заголовком: хеш сойдётся только под своим H5.
2. Сектор, пришедший **раньше** своего заголовка, при первом проходе не опознаётся (H5 ещё неизвестен). Он добирается позже адресной перепривязкой удержанных данных (раздел 8).
3. Число томов N выводится из H2 (`ceil(FileSize / 64)` , минимум 1), поэтому H4 задаёт полное пространство номеров: секторы нумеруются `0 .. N+M-1` (сначала N data-томов, затем M ECC-томов).

3. Кодирование файла

`FileEncoder.Encode(content, fileName)` выполняет следующие шаги (шкала прогресса: подготовка 0–10, ECC 10–75, пакеты 75–100):

1. **Валидация.** Размер $\leq 16\,777\,215$ байт (3-байтное поле H2); имя упаковывается в поле H1 (14 байт): база усекается с маркером «~», расширение сохраняется целиком; непредставимое имя — отказ.
2. **SHA-256** содержимого — будущая верификация при сборке.
3. **Разбивка на data-тома:** $N = \max(1, \lceil \text{size} / 64 \rceil)$; последний том добивается нулями.
4. **Число ECC-томов:** $M = \max(1, \lceil N \cdot \text{eccPercent} / 100 \rceil)$ при $\text{eccPercent} \geq 1$, иначе 0. Ограничение поля: $N + M \leq 65\,535$.
5. **RS-кодирование:** N data-томов $\rightarrow M$ ECC-томов (раздел 4).
6. **Заголовок:** сериализация H1–H4 (51 байт) $\rightarrow \text{Trunc24}(\text{SHA-256}) \rightarrow$ пакет заголовка. Значение H5 запоминается.
7. **Секторы:** для каждого тома $i \in [0, N+M)$ — пакет `seqNum(2) + payload(64) + hash(9)`, где хеш вычисляется с сидом H5.
8. **Расстановка в потоке:** копий заголовка $H = \max(3, \lceil (N+M) \cdot \text{headerPercent} / 100 \rceil)$; первый и последний пакеты потока — заголовки, промежуточные копии вставляются равномерно, каждые `max(1, (N+M) / (H-1))` секторов.
9. **Вайп** промежуточных буферов (data/ECC payloads, сериализованный заголовок).

Результат — список 75-байтных пакетов. Текстовое представление (`EncodeToText`) — по одному пакету на строку Base64.

3.1. Детали расстановки заголовков

`ArrangePackets` строит поток так: первый пакет — заголовок; далее секторы по порядку номеров, и после каждого `interval`-го сектора вставляется промежуточная копия, где `interval = max(1, (N+M) / (H-2+1))`; последний пакет — снова заголовок. Целочисленное деление намеренное: при малых T копии стремятся к началу потока, что безопаснее для случая обрезки хвоста. Итоговое число копий всегда равно H (промежуточные вставляются, пока не исчерпан счётчик `H-2`). Порядок секторов в потоке естественный (0, 1, ..., $N+M-1$) — декодеру порядок безразличен, а предсказуемость удобна для отладки и склеивания потоков.

3.2. Сквозной пример кодирования

Файл `report.txt` из 100 байт, ECC 10%, заголовки 3%:

1. Имя `report.txt` (9 ASCII-байт) уместается в H1 без усеечения — добивается пробелами до 14.
2. $N = \lceil 100/64 \rceil = 2$; том D1 содержит байты 64..99 плюс 24 нуля добивки.
3. $M = \max(1, \lceil 2 \cdot 10/100 \rceil) = 1$; $T = N+M = 3 \leq 65\,535$ — поле хватает.
4. RS: для каждой из 32 позиций символа s строится столбец `[d0s, d1s, e0s]`; `Process` вычисляет $e0^s$ — символ ECC-тома (коэффициенты фиксированы матрицей кода RsRaid16).

5. Заголовок: $H_1(14) + H_2(3: \boxed{100} \rightarrow \boxed{64\ 00\ 00}) + H_3(32: \text{SHA-256 файла}) + H_4(2: \boxed{01\ 00}) \rightarrow 51$ байт $\rightarrow H_5 = \text{Trunc}_{24}(\text{SHA-256}(\dots)) \rightarrow 75$ -байтный пакет H .
6. Секторы: D_1 -пакеты для номеров 0 (данные), 1 (данные), 2 (ECC); хеш каждого = $\text{Trunc}_9(\text{SHA-256}(H_5 \parallel \text{номер} \parallel \text{payload}))$.
7. $N = \max(3, \lceil 3 \cdot 3 / 100 \rceil) = 3$; $\text{interval} = \max(1, 3/2) = 1 \rightarrow$ копия после первого сектора.
8. Поток (6 пакетов): H, D_0, H, D_1, D_2, H .
9. Вайп: буферы data/ECC payload и сериализованный заголовок очищаются; живут только возвращённые пакеты.

4. Избыточность: стирающий код Рида-Соломона

Реализация поля $\text{GF}(2^{16})$ — `RsRaid16` (не изменяется; демо — `RsRaid16Demo`). Адаптер `RsCodeAdapter` работает с 64-байтными томами:

- Один символ поля = 2 байта (UInt16 LE). Том = **32 независимых GF-символа**.
- Кодирование:** для каждой из 32 позиций символа собирается столбец из K data-символов, `RsRaidBase.Process` достраивает M ECC-символов в хвосте столбца, результат пишется в ECC-тома. Каждый ECC-том — линейная комбинация data-томов над полем.
- Декодирование (стирания, не ошибки):** на вход подаётся $K+M$ слотов с картой валидности. Условие восстановления — **число стёртых data-томов $\leq$ числу доступных ECC-томов**. Матрица системы обращается с учётом карты стираний; стёртые символы вычисляются позиция за позицией.
- Предел: $K + M \leq 65\ 535$ (размер поля).

Почему стирающий, а не исправляющий ошибки: целостность каждого принятого тома уже гарантирована усечённым SHA-256 (вероятность пропустить подмену $\sim 2^{-72}$). Кодеку достаточно восстанавливать **пропуски**, а не исправлять искажения, — это радикально проще и быстрее. Практический предел размера файла определяется полем, а не H_2 : без ECC $N \leq 65\ 535 \rightarrow \sim 4,19$ МБ; при $\text{eccPercent} = 10\%$ $N \cdot 1.1 \leq 65\ 535 \rightarrow \sim 3,8$ МБ.

4.1. Постатейная механика кодирования ($K=2, M=1$)

Тома D_0, D_1 и E_0 — по 32 GF-символа (UInt16 LE). Кодирование обходит позиции $s = 0..31$ независимо:

Позиция s	Столбец до <code>Process</code>	Столбец после <code>Process</code>
0	$[d_0^0, d_1^0, ?]$	$[d_0^0, d_1^0, e_0^0]$
1	$[d_0^1, d_1^1, ?]$	$[d_0^1, d_1^1, e_0^1]$
...	...	...
31	$[d_0^{31}, d_1^{31}, ?]$	$[d_0^{31}, d_1^{31}, e_0^{31}]$

Каждый ECC-символ — одна и та же линейная форма от K data-символов столбца с коэффициентами матрицы кода; матрица строится `RsRaid16.Init(k, m, null)` и не зависит от

данных. Восстановление симметрично: `Init(k, m, validityMap)` строит систему только по валидным слотам, и `Process` решает её для каждого столбца отдельно. Для адаптера `RsRaid16` — чёрный ящик с контрактом «Init → Process по столбцам»; сами вычисления поля (таблицы логарифмов, обращение матрицы) — в немодифицируемом справочнике `refs-src/RsRaid16.cs`. Стоимость: кодирование — 32 столбца × K сложений/умножений поля на каждый ЕСС-символ; восстановление — однократное обращение матрицы при `Init` плюс 32 столбца × решение системы. Объём работы линеен по размеру файла.

5. Конвейер декодирования

```
источник байт (Source)
    |
    ▼ (только txt-режим: отбрасывает всё вне алфавита Base64)
фильтр ByteRangeFilter
    |
    ▼ (скользящее окно с шагом 1 байт; удерживает весь поток)
сканер SlidingWindowScanner
    | EmitPacket: только распознанные 75-байтные пакеты
    ▼
накопитель StreamProcessor → ReceptionSlot × количество файлов
    |
    ▼ (по запросу: TryAssemble)
сборка: прямая → RS → обход коллизий версий
```

- **Источник** (`ByteArraySource`, `StreamSource`, `FileSource`) — событийная модель с буфером и приостановкой чтения (см. `DeveloperGuide.ru.md`).
- **Фильтр** включён только в txt-режиме: переводит поток в «плотный» Base64-байтовый поток, отбрасывая переводы строк, пробелы и мусор. В binary-режиме фильтр исключается из цепочки.
- **Сканер** — см. раздел 6.
- **Накопитель** — см. раздел 7.

Поддерживаются два формата входа: **Base64-текст** (окно 100 байт) и **сырые пакеты** (окно 75 байт). Сканеры обоих форматов создаются лениво и живут одновременно: смешанный поток (текст + вставленные двоичные куски) обрабатывается последовательными вызовами `Scan`, удержанные данные сохраняются между вызовами.

6. Распознавание в шуме: скользящее окно

`SlidingWindowScanner` применяет к позициям потока делегат окна `WindowHandler(ReadOnlySpan<byte> window, out byte[]? emitted) → int`. Возвращённое делегатом значение — продвижение **прямого прохода**:

- окно в позиции p распознано → возвращается `emitted` (копия пакета) и продвижение на длину пакета (75 или 100 байт);
- не распознано → продвижение **на 1 байт**, окно сдвигается.

Прыжок на длину пакета после успеха — оптимизация прямого прохода. Он может перепрыгнуть начало другого валидного пакета, перекрывающегося с только что принятым (реально встречается на повреждениях: фрагмент одного тома вместе с первыми символами соседней строки случайно декодируется в валидный пакет). Полнота покрытия от прыжков не зависит: повторный проход перепривязки (раздел 8) проверяет удержанные данные на **каждой** позиции. Логика окна (txt-режим `FileDecoder.TxtWindow`): декодировать 100 Base64-символов в 75 байт (неудача декода → сдвиг на 1); спросить накопитель `Recognizes(packet)` (побочных эффектов нет) — это автономный хеш заголовка **или** хеш сектора под один из известных заголовков. Успех → выдать пакет.

Так рассинхронизация, шум и переводы строк между пакетами обходятся автоматически: окно «нащупывает» следующую границу пакета побайтовым сдвигом.

6.1. Трассировка прямого прохода (txt-режим, фрагмент)

```
поток: ...мусор...[100 Base64-символов пакета][мусор]...
позиция p, окно [p, p+100):
    p      — окно начинается в мусоре: декод/распознавание неуспешно → продвижение +1
    p+1    — то же → +1
    ...    — побайтовый сдвиг, пока окно не встанет на границу пакета
    p+k    — окно = пакет: Base64 → 75 Б, Recognizes = true
            → EmitPacket(копия), продвижение +100
    p+k+100— окно после пакета (мусор) → +1, поиск продолжается
```

Стоимость прямого прохода — $O(\text{длина потока})$ вызовов окна в худшем случае; на плотном потоке (пакеты подряд) — $O(\text{длина}/75)$ за счёт прыжков. Прыжок +100 после успеха может перепрыгнуть начало пакета, перекрывающегося с принятым (реально встречается на повреждениях) — такие потери закрывает исчерпывающий повторный проход перепривязки (раздел 8): он не прыгает вовсе.

Сканер **удерживает весь обработанный поток** (объёмы входа малы) — это плата за возможность адресной перепривязки (раздел 8): данные, не опознанные при первом проходе, не теряются никогда.

7. Накопление секторов

Двухуровневая структура: `StreamProcessor` — внешний контур (поток кусков → пакеты → слоты), `ReceptionSlot` — элементарный накопитель одного файла.

7.1. StreamProcessor: от кусков к пакетам

Вход приходит произвольными кусками. Буфер `_pending[75]` склеивает куски: в цикле собираются байты до полного пакета; неполный хвост остаётся в `_pending` до следующего куска. Полный пакет клонируется и проходит `AcceptPacket`:

1. **Автономный хеш сошёлся** → пакет-заголовок → `AcceptHeader`.
2. **Иначе** → попытка принять как сектор **в каждый** известный слот (`AcceptSectorForSlot`): номер в диапазоне `[0, N+M)` и хеш с сидом H5 этого слота сходится. Один пакет теоретически может быть валидным сектором нескольких слотов — принимается во все.

Распознанный пакет эмитится дальше по конвейеру (`EmitPacket` — неделимые порции). Новые слоты собираются в список, и **после выхода из блокировки** выбрасывается `HeaderAccepted(header, headerHash)` на каждый — сигнал вышестоящим модулям сделать адресную перепривязку (раздел 8).

Приём заголовка `AcceptHeader`:

- побайтовое сравнение с каждым существующим слотом → совпадение означает ещё одну копию: `IncrementHeaderCount()` (с насыщением `int.MaxValue`), нового слота нет;
- иначе — десериализация `HeaderContent`, чтение H5 из пакета, создание `ReceptionSlot` и добавление в список.

Потокобезопасность: приём и чтение состояния (снимки `Slots`, счётчики) — под одной блокировкой; события — вне её.

7.2. ReceptionSlot: слот одного файла

Внутреннее состояние:

```
SortedDictionary<int sectorNum, List<SectorVariant>> _sectors
```

- ключ — номер сектора `[0, N+M)`;
- значение — **версии** payload; `SectorVariant` = payload(64) + `ConfirmationCount` (сколько копий этой версии принято);
- список версий **всегда отсортирован по убыванию счётчика** — первый элемент самый подтверждённый.

`AddSector(sectorNum, payload)` по шагам:

1. Номер вне `[0, N+M)` или длина $\neq 64$ → отказ.
2. Первая версия сектора? Создать список, добавить версию со счётчиком 1.
3. Линейный поиск версии с **побайтово равным** payload:
 - найдена → `ConfirmationCount++` (с насыщением), затем **всплытие**: версия поднимается вверх, пока счётчик соседа сверху строго меньше. Через равные счётчики не переставляется — это сохраняет текущий порядок равновероятных элементов (важно для эвристики сборки, раздел 9.7).

- не найдена → новая версия добавляется **в конец** списка (все существующие имеют счётчик ≥ 1 , поэтому сортировка не нарушается).

Почему версии, а не «последний принятый выигрывает»: канал может дублировать и задерживать пакеты. Если две разные копии одного номера прошли проверку хеша (событие крайне маловероятное, но конечное — либо целенаправленная подделка), накопитель сохраняет **обе** со счётчиками. Приём идёт без потерь информации: решение о том, какая версия правильная, откладывается до сборки, где есть независимый арбитр — SHA-256 всего файла. Метрики слота: `HeaderReceptionCount` (копии заголовка), `ReceivedSectorCount` (номера $s \geq 1$ версией), `ReceivedSectorCopyCount` (все копии, сумма счётчиков), `CollisionSectorCount` (номера $s > 1$ версией), `Coverage` (доля принятых номеров от $N+M$), `BuildValidityMap()` (■/■/■-карта: принят/коллизия/пропущен), `BuildCollisionMap()` (номер → кратность коллизии), `GetSectorVersions(n)` (снимок версий в порядке предпочтительности).

7.3. Многофайловость

Каждый уникальный заголовок порождает свой слот. Привязка хеша к H_5 не позволяет сектору «утечь» в чужой слот: даже одинаковые номера томов разных файлов различаются H_5 . Сборка (`FileDecoder.TryAssemble(header)`) ищет слот побайтовым сравнением сериализованного заголовка и собирает именно его.

8. Адресная перепривязка: сектор раньше заголовка

Проблема: сектор, пришедший до первого заголовка своего файла, при первом проходе не опознается — H_5 ещё неизвестен, а проверка хеша невозможна (автономный хеш заголовка над D_1-D_2 почти наверняка не сойдётся). Сканер сдвинется на 1 байт и пойдёт дальше.

Решение — по событию `HeaderAccepted` декодер просит **оба** сканера (`txt` и `bin`) выполнить `RequestRescan` специальным окном `RebindWindow`:

1. Окно декодируется как обычно (`Base64` → 75 байт или сырые 75 байт).
2. Номер сектора проверяется на диапазон `[0, N+M)` нового заголовка.
3. Проверяется хеш сектора с сидом = H_5 **нового** заголовка.
4. Успех → пакет эмитится (принимается накопителем как сектор этого слота), неудача → ничего; окно в обоих случаях сдвигается на 1 байт — продвижение, возвращённое делегатом окна, повторным проходом игнорируется.

Повторный проход покрывает удержанные данные **от начала потока до границы последней прямой выдачи** и продвигается побайтово на каждой позиции, игнорируя продвижение, возвращённое окном: прямой проход после успеха прыгает на длину пакета и может перепрыгнуть начало перекрывающегося валидного пакета (раздел 6) — только исчерпывающий проход без пропусков закрывает такие потери. Дальше границы прямой проход ещё не дошёл, и повторная проверка тех же позиций продолжится обычным сканированием. Так исключается двойное подтверждение одних и тех же секторов сверх одной перепривязки.

Если сканер другого формата простаивает (вызовы `Scan` последовательны), накопитель временно перепоподключается к нему на время синхронной перепривязки, затем возвращается к активному сканеру. Промежуточные перепривязки нескольких заголовков не конфликтуют: каждое окно проверяется только по своему H5, подтверждения остальных слотов не затрагиваются.

9. Сборка файла

Сборка (`ReceptionSlot.TryAssemble`) — самый насыщенный алгоритм кодека. Арбитр подлинности — SHA-256 всего файла из H3. Ниже — рабочий обзор; формальное изложение (инварианты структур, точные определения пространства поиска, математика одометра и прокрутки, проработанные примеры, границы гарантий) вынесено в `AssemblyGuide.Academic.ru.md` (инженерный пересказ — `AssemblyGuide.Engineer.ru.md`).

9.1. Общий план

`TryAssemble`

- | выбранная комбинация = первые (самые подтверждённые) версии каждого сектора
- | точки ветвления = секторы с равновероятными версиями (`ChoicePoint`)
- |
- | точек ветвления нет → одна попытка: прямая сборка → RS
- |
- | точки есть:
 - 1. попытка №1 (нулевая комбинация) с замером времени
 - 2. число комбинаций C = произведение кратностей равновероятности
 - 3. оценка времени = $t(\text{попытки}) \cdot C$
 - 4. $C \leq 100\,000$ и оценка ≤ 30 с → полный перебор (9.6)
 - иначе → эвристическая прокрутка (9.7)
- |
- | результат L и $M > 0$ → подбор подмножества томов (9.8)

9.2. Одна попытка: прямая сборка

`BuildPreferredSelection()` — массив длины $N+M$; для каждого принятого номера берётся `versions[0]` (самый подтверждённый payload). Прямая сборка succeeds, если **все** N **data-томов** присутствуют: payloads склеиваются в буфер $N \cdot 64$, буфер обрезается до `FileSize` и проверяется SHA-256 (9.9).

9.3. Одна попытка: RS-восстановление

Прямая не прошла, а `EccCount > 0`:

1. Строится карта валидности длины $N+M$ (том принят/стёрт) и массив слотов с выбранными payload.
2. Если принято томов меньше N — восстановить нечем, отказ.
3. `RsCodecAdapter.Decode(sectors, map, N)` : passthrough, если data целы; отказ, если стёртых data больше, чем доступных ECC; иначе — решение системы, стёртые data-тома восстанавливаются символ за символом.
4. Восстановленные N томов склеиваются, обрезаются, проверяются SHA-256.

Заметьте: RS-восстановление уже включено в **каждую** попытку комбинационного поиска — перебор варьирует только неоднозначные секторы, а недостающие тома каждый раз добиваются ECC.

9.4. Точки ветвления

`ChoicePoint` — сектор, у которого несколько версий делят **максимальный** счётчик подтверждений (`TiedVariantCount` ≥ 2). Версии с меньшими счётчиками в ветвлении не участвуют: они заведомо хуже подтверждены и не выбираются ни одной стратегией поиска в текущем вызове — если истина оказалась в меньшинстве подтверждений, сборка честно откажет (сценарии и компромисс разобраны в `AssemblyGuide.Academic.ru.md`, разделы 8 и 16).

9.5. Выбор стратегии

`SectorCombinationMath.CountCombinations(factors)` — произведение кратностей с насыщением на `long.MaxValue` (переполнение распознаётся и выводится как `<> long.Max`). Первая (нулевая) попытка собирается с замером времени; оценка полного перебора = время попытки $\times C$. `ShouldUseExhaustiveSearch(C, оценка, options)` требует одновременно: $C \leq \text{MaxExhaustiveCombinations}$ (100 000) и оценка $\leq \text{TimeBudget}$ (30 с). Лимиты настраиваются `SectorVersionSearchOptions`.

9.6. Полный перебор (exhaustive)

Декартов перебор всех комбинаций равновероятных версий. Индексы выбора хранятся массивом `indexes[]` с модулями `moduli[]` (кратности точек ветвления); `AdvanceIndexes` — классический **одометр**: младшая позиция справа, инкремент с переносом, при полном исчерпании — false и обнуление. Нулевая комбинация уже проверена, цикл идёт с 1 до $C-1$: применить индексы к выбранному набору (`ApplyIndexes`), собрать (9.2 $\rightarrow$ 9.3), при успехе вернуть. На каждой итерации — проверка отмены и бюджета времени 30 с.

9.7. Эвристическая прокрутка (rotation)

Когда перебор неразумен (C велико), используется наблюдение: после каждой неудачной попытки **во всех** неоднозначных слотах первый равновероятный вариант переносится в конец равновероятной части (`RotateTiedVariants` ; версии с меньшими счётчиками не затрагиваются — сортировка остаётся корректной).

Пример: слот с версиями A/B/C (счётчики равны) и слот X/Y дают последовательность попыток:

$A+X \rightarrow B+Y \rightarrow C+X \rightarrow A+Y \rightarrow B+X \rightarrow C+Y$

Число уникальных состояний синхронной прокрутки — **НОК длин циклов** (`CountRotationStates`), ограниченный сверху `MaxHeuristicAttempts` (100 000). Для примера: $\text{НОК}(3,2) = 6$ — все 6 комбинаций действительно покрываются. При кратностях-простых между собой $\text{НОК} = C$ и прокрутка эквивалентна полному перебору; при общих делителях часть комбинаций недостижима (это цена эвристики), но наиболее вероятные «систематические» подделки (одинаковый сдвиг во всех слотах) она обрабатывает. Бюджет времени и отмена проверяются на каждом состоянии.

9.8. Подбор подмножества томов

Стадия 3 покрывает **тихие повреждения**: сектор испорчен, но номер цел и усечённый хеш D3 сошёлся, версия одна — точки ветвления нет, а RS по полной карте принимает испорченный том за истину. `SubsetMaskPlanner` строит по уровням маски исключения: база — все коллизийные слоты (исключаются и восстанавливаются целиком), уровень 1 — одиночные исключения присутствующих томов по подозрительности (менее подтверждённые раньше, равные — псевдослучайно с сидом от H5), уровни 2–3 — пары и тройки из шорт-листа. Попытка — RS-восстановление исключённых по оставшимся и финальный SHA-256; лимиты — кап на стёртые data-тома ($E \leq 32$), 100 000 попыток, 30 с. Формальный разбор — `AssemblyGuide.Academic.ru.md` (раздел 12).

9.9. Верификация результата

`TrimAndVerify`: буфер N·64 обрезается до `FileSize` (последний том добивался нулями), вычисляется SHA-256 и сравнивается с H3. Несовпадение — буферы очищаются, попытка считается неудачной. Совпадение — файл возвращается: сборка криптографически доказательна, «частично правильных» результатов не бывает.

10. Рандомизированный тестовый стенд

`DataShield.Demo` — непрерывный стенд стабильности и производительности:

- вход 1 байт ... 256 КБ (лог-равномерно), ECC 1–200%;
- с вероятностью 10% — многофайловый поток, 3% — пустой файл;
- каждый прогон повреждается маской из `DamageBits` (потери секторов сверх бюджета ECC, обрезки, мусор, перестановки, дубли, подделки-победители и др.);
- **PASS** — повреждение в бюджете, восстановлено бит-в-бит; **WARN** — намеренное повреждение сверх бюджета/подделка: отказ кодека корректен; **FAIL** — реальный дефект кодека;
- таблица последних итераций с накопленной статистикой и скоростями кодирования/декодирования.

Стенд крутится бесконечно и является основным интеграционным инструментом регрессии: любое FAIL-поведение — баг.

11. Числовая сводка

Параметр	Значение
Размер пакета / Base64	75 байт / 100 символов без паддинга
Payload тома	64 байта = 32 символа GF(2^16)
Поля заголовка	имя 14 Б + размер 3 Б + SHA-256 + M 2 Б + H5 24 Б
Максимум файла (поле H2)	16 777 215 байт
Максимум томов (поле GF)	$N + M \leq 65\,535$ (~4,19 МБ данных без ECC)
Избыточность по умолчанию	ECC 10%, заголовки 3% (минимум 3 копии)
Контрольные суммы	усечённый SHA-256 (24 Б / 9 Б); верификация — SHA-256
Лимиты поиска версий	100 000 комбинаций/состояний, 30 с
Лимиты подбора томов	100 000 масок, $E \leq 32$, 30 с